



ESTUDIANTE: Mario David Pilco Flores

CARRERA: Ingeniería en software

PARALELO: B

ASIGNATURA: Algoritmos y Lógica de Programación **NIVEL:** 1 **FECHA:** 12/05/2026

DOCENTE: Ing. José Caiza

TEMA: Persistencia con archivos de texto

LINK DEL GITHUB: [Algoritmos-y-l-gica-de-programaci-n/T2.1 Persistencia con archivos de texto](https://github.com/mariopilco321/Algoritmos-y-l-gica-de-programaci-n/T2.1%20Persistencia%20con%20archivos%20de%20texto) at main · mariopilco321/Algoritmos-y-l-gica-de-programaci-n

I. INTRODUCCIÓN

La persistencia de datos es una técnica fundamental en programación, ya que permite almacenar información de manera permanente para ser utilizada posteriormente. En el lenguaje C++, el manejo de archivos de texto facilita guardar y recuperar datos mediante el uso de librerías especializadas como `fstream`.

En esta práctica se desarrolló un sistema básico para la gestión de registros de estudiantes, permitiendo almacenar información como nombre, edad, carrera y promedio en un archivo de texto. Además, se aplicaron validaciones y estructuras de control para garantizar el correcto funcionamiento del programa.

II. OBJETIVO GENERAL

Desarrollar un programa en C++ que permita almacenar información de forma permanente utilizando archivos de texto.

III. OBJETIVOS ESPECÍFICOS

- Implementar un sistema de registro de estudiantes utilizando archivos de texto en C++.
- Aplicar validaciones de datos y un menú interactivo para mejorar el funcionamiento del programa.

IV. DESARROLLO DEL PROGRAMA

Enunciado: Desarrollar un sistema en C++ que permita gestionar registros de estudiantes en un archivo de texto.

1. ANALISIS

El programa permite registrar información de estudiantes y almacenarla de manera permanente en un archivo de texto llamado `estudiantes.txt`. El sistema utiliza un menú interactivo donde el usuario puede elegir entre registrar estudiantes o salir del programa.

Para cada estudiante se solicitan los siguientes datos:

- Nombre
- Edad
- Carrera
- Promedio

El programa realiza validaciones para evitar:

Edades menores o iguales a cero.

Promedios fuera del rango de 0 a 10.

Los datos ingresados se guardan utilizando archivos de texto mediante `ofstream` y el modo `ios::app`, el cual permite agregar información sin borrar los registros anteriores.

2. DECLARACION DE VARIABLES

Variables de Entrada



nombre : cadena

Almacena el nombre del estudiante ingresado por el usuario.

edad : entero

Guarda la edad del estudiante.

carrera : cadena

Almacena la carrera del estudiante.

promedio : real

Guarda el promedio académico del estudiante.

opcion : entero

Permite seleccionar una opción del menú interactivo.

Variables de Proceso

archivo : ofstream

Variable utilizada para abrir, escribir y guardar información en el archivo de texto.

Variables de Salida

Mensaje: "Datos guardados correctamente"

Indica que la información fue almacenada en el archivo.

Mensaje: "Error al abrir el archivo"

Se muestra cuando ocurre un problema al crear o abrir el archivo.

Mensaje: "Opcion invalida"

Aparece cuando el usuario ingresa una opción incorrecta en el menú.

Archivo estudiantes.txt

Contiene todos los registros guardados de los estudiantes.

3. ALGORITMO

- Iniciar el programa.
- Mostrar el menú de opciones.
- Leer la opción seleccionada.
- Si la opción es registrar estudiante:
 - Pedir nombre.
 - Pedir edad y validar que sea mayor a 0.
 - Pedir carrera.
 - Pedir promedio y validar que esté entre 0 y 10.
- Abrir el archivo en modo agregar.
- Guardar los datos en el archivo.
- Cerrar el archivo.
- Mostrar mensaje de éxito.
- Si la opción es salir:
 - Mostrar mensaje de despedida.
- Si la opción es inválida:
 - Mostrar mensaje de error.
- Repetir el proceso hasta elegir salir.

4. PSEUDOCÓDIGO

Algoritmo Registro_estudiantes

Definir opcion Como Entero

Definir nombre, carrera Como Cadena

Definir edad Como Entero

Definir promedio Como Real



Repetir

Escribir "===== MENU ====="

Escribir "1. Registrar estudiante"

Escribir "2. Salir"

Escribir "Seleccione una opcion: "

Leer opcion

Si opcion = 1 Entonces

Escribir "Ingrese el nombre del estudiante:"

Leer nombre

Repetir

Escribir "Ingrese la edad:"

Leer edad

Si edad ≤ 0 Entonces

Escribir "Error: Edad invalida"

FinSi

Hasta Que edad > 0

Escribir "Ingrese la carrera:"

Leer carrera

Repetir

Escribir "Ingrese el promedio:"

Leer promedio

Si promedio < 0 O promedio > 10 Entonces

Escribir "Error: Promedio invalido"

FinSi

Hasta Que promedio ≥ 0 Y promedio ≤ 10

Escribir "Datos guardados correctamente"

Sino

Si opcion = 2 Entonces

Escribir "Saliendo del programa..."

Sino

Escribir "Opcion invalida"

FinSi

FinSi

Hasta Que opcion = 2

FinAlgoritmo

5. DIGRAMA DE FLUJO

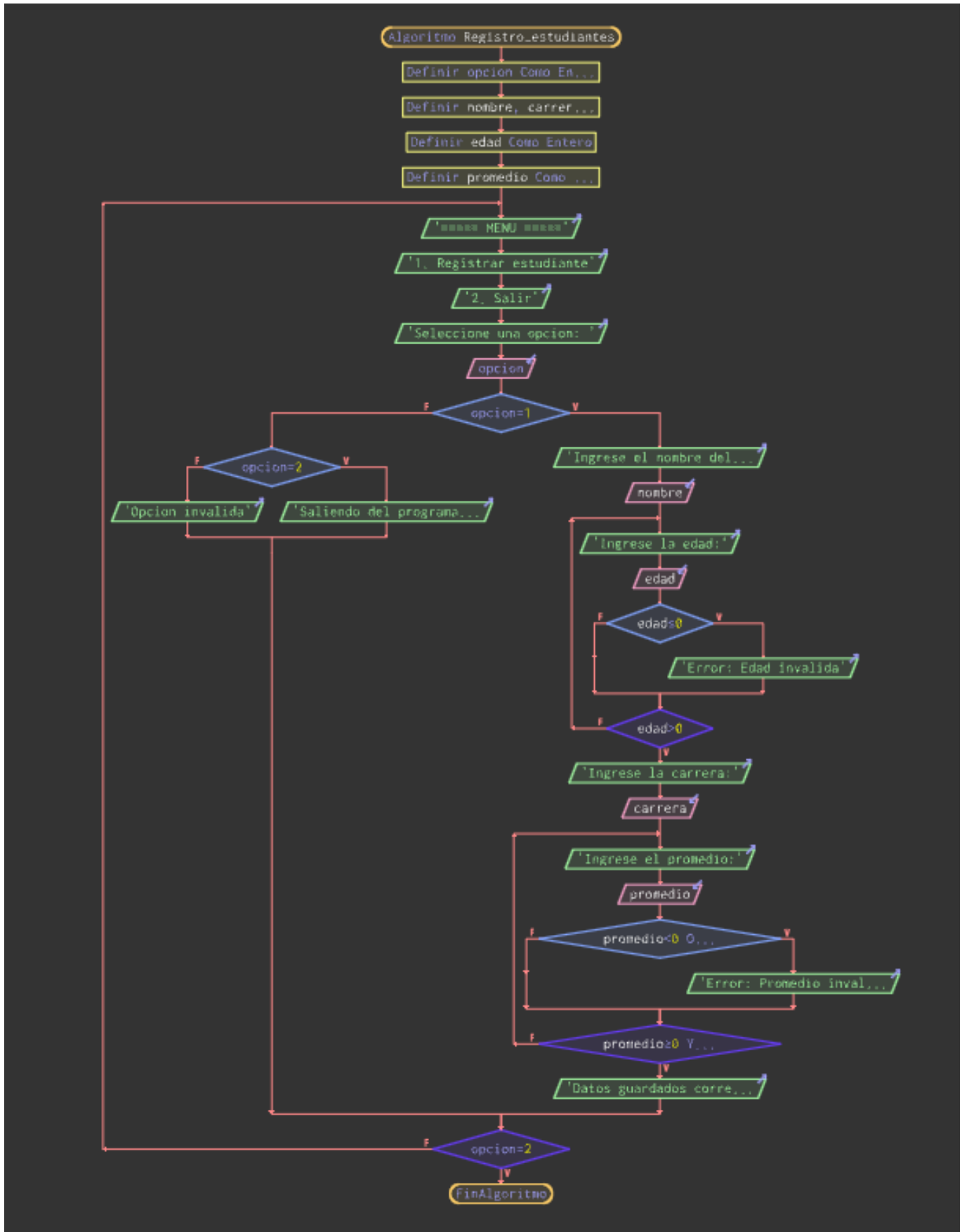


Imagen 1: Diagrama de Flujo
 Elaborado por: Mario Pilco



6. CÓDIGO EN C++

```
#include <iostream> // Librería para entrada y salida de datos
#include <fstream>   // Librería para manejo de archivos
#include <string>     // Librería para utilizar cadenas de texto
```

```
using namespace std;
```

```
int main() {
```

```
    // Variable para manejar el archivo
    ofstream archivo;
```

```
    // Variable para guardar la opción del menú
    int opcion;
```

```
    // Ciclo principal del programa
    do {
```

```
        // Mostrar menú
        cout << "\n===== MENU =====" << endl;
        cout << "1. Registrar estudiante" << endl;
        cout << "2. Salir" << endl;
        cout << "Seleccione una opcion: ";
        cin >> opcion;
```

```
        // Opción para registrar estudiante
        if(opcion == 1) {
```

```
            // Variables para almacenar datos del estudiante
            string nombre, carrera;
            int edad;
            float promedio;
```

```
            // Limpia el buffer del teclado
            cin.ignore();
```

```
            // Solicita el nombre del estudiante
            cout << "\nIngrese el nombre del estudiante: ";
            getline(cin, nombre);
```

```
            // Validación de la edad
            do {
```

```
                cout << "Ingrese la edad: ";
                cin >> edad;
```

```
                // Verifica que la edad sea válida
                if(edad <= 0) {
                    cout << "Error: Edad invalida." << endl;
                }
```

```
            } while(edad <= 0);
```



```
// Limpia nuevamente el buffer
cin.ignore();

// Solicita la carrera
cout << "Ingrese la carrera: ";
getline(cin, carrera);

// Validación del promedio
do {

    cout << "Ingrese el promedio: ";
    cin >> promedio;

    // Verifica que el promedio esté entre 0 y 10
    if(promedio < 0 || promedio > 10) {
        cout << "Error: Promedio invalido." << endl;
    }

} while(promedio < 0 || promedio > 10);

// Abrir archivo en modo agregar
archivo.open("estudiantes.txt", ios::app);

// Verifica si el archivo se abrió correctamente
if(archivo.is_open()) {

    // Guardar información en el archivo
    archivo << "Nombre: " << nombre << endl;
    archivo << "Edad: " << edad << endl;
    archivo << "Carrera: " << carrera << endl;
    archivo << "Promedio: " << promedio << endl;
    archivo << "-----" << endl;

    // Cerrar archivo
    archivo.close();

    // Mensaje de confirmación
    cout << "\nDatos guardados correctamente." << endl;

} else {

    // Mensaje de error si no se abre el archivo
    cout << "Error al abrir el archivo." << endl;
}

}

// Opción para salir
else if(opcion == 2) {

    cout << "\nSaliendo del programa..." << endl;
```



```
}  
  
// Si el usuario ingresa una opción incorrecta  
else {  
  
    cout << "\nOpcion invalida." << endl;  
}  
  
} while(opcion != 2); // Repetir hasta seleccionar salir  
  
return 0; // Fin del programa  
}
```

7. PRUEBAS DE ESCRITORIO

```
===== MENU =====  
1. Registrar estudiante  
2. Salir  
Seleccione una opcion: 1  
  
Ingrese el nombre del estudiante: Pilco Mario  
Ingrese la edad: 18  
Ingrese la carrera: Software  
Ingrese el promedio: 9.10
```

Imagen 2: Prueba de escritorio C++ #1
Elaborado por: Mario Pilco

```
===== MENU =====  
1. Registrar estudiante  
2. Salir  
Seleccione una opcion: 1  
  
Ingrese el nombre del estudiante: Atiencia Josue  
Ingrese la edad: 18  
Ingrese la carrera: Software  
Ingrese el promedio: 8  
  
Datos guardados correctamente.
```

Imagen 3: Prueba de escritorio C++ #2
Elaborado por: Mario Pilco

```
===== MENU =====  
1. Registrar estudiante  
2. Salir  
Seleccione una opcion: 2  
  
Saliendo del programa...  
  
Process returned 0 (0x0)   execution time : 107.326 s  
Press any key to continue.
```

Imagen 4: Prueba de escritorio C++ #3
Elaborado por: Mario Pilco



V. CONCLUSIONES

- El uso de archivos de texto en C++ permite almacenar información de manera permanente y organizada.
- Las validaciones y el menú interactivo ayudan a mejorar la seguridad y funcionalidad del programa.

VI. RECOMENDACIONES

- Implementar funciones para organizar mejor el código y facilitar su mantenimiento.
- Agregar opciones para consultar, modificar o eliminar registros almacenados.
- Realizar pruebas constantes para verificar que las validaciones funcionen correctamente.